Architecture & Trade-offs

ledger-core: an in-memory account ledger core

1. Append-only at scale

The current implementation's costliest habit is `AccountLedger.balanceAsOf()`: every call re-scans and re-sums the *entire* entries array from account inception, filtered by value date and posted-day. It's called once per account per day-close, plus ad hoc for point-in-time queries. At six days and two accounts this is free. At 100x transaction volume, sustained over years rather than days, two things break in order.

First to break: the daily close. A nightly batch that re-sums an account's whole lifetime history to compute one day's closing balance is O(total lifetime entries) per account per day, forever; a bank's oldest, most active accounts get slower to close every single day, with no ceiling. This is the classic general-ledger mistake of treating "closing balance" as a query instead of a maintained value.

Second, and structural: unbounded in-process memory. Every entry ever posted, and every hold ever created (`ACTIVE`, `SETTLED`, or `DECLINED`), lives in a JS array/map for the process's entire lifetime; nothing is ever evicted, archived, or paged out. That's an explicit, correct simplification for a six-day, in-memory exercise, but it's also exactly where "in-memory" stops being viable: at 100x volume sustained over real account lifetimes, per-account memory grows linearly and without bound, and a single long-lived process eventually cannot hold a large book in RAM regardless of how fast the CPU is.

Cheapest structural fix: stop re-deriving the closing balance from scratch and start *carrying it forward*. Cache a balance-forward snapshot each time a day closes (balance as of day N, plus the posted-day watermark it was computed at), and require every subsequent query to read that snapshot plus only the entries posted *since* it, never the full history again. This is precisely how real general ledgers close monthly: detail is retained for audit, but the hot path only ever touches the delta since the last checkpoint. It requires no new infrastructure (no database, no external cache); just treating "closing balance" as a value that gets computed once and carried, instead of a query re-run from day zero every time it's asked.

2. Value-dated entries in production

Backdating (E7, booked Day 5, value-dated Day 2) is the single feature in this exercise with the largest regulatory surface, because it lets the *meaning* of a past closing balance change after that day has already been reported to a customer and, potentially, to the regulator. Three concrete exposures follow directly from that:

- **Statement integrity.** A UAE-licensed bank sends customers periodic statements. If a Day 2 balance can silently mean something different once queried after Day 5 posts a backdated entry, "what my statement said" and "what the system now says happened" diverge, and a customer dispute has to be resolved by reconstructing *which value was true when disclosed*, not just what's true now.
- **AML/CTF timing manipulation.** The ability to book an arbitrary past value_date is a natural vector for making a transaction appear to fall outside a monitoring or reporting window, or for reordering cash flows around a reporting cutoff; exactly the pattern CBUAE-aligned AML controls (and FATF-derived transaction-monitoring rules generally) are built to catch. An unconstrained backdating capability is a control gap by itself, independent of whether any single entry is malicious.

- **Regulatory reporting cutoffs.** Month-end and period-end regulatory submissions (liquidity, capital, large-exposure reporting) are computed from closing balances as of a cutoff. A value-dated entry landing after a cutoff has already been reported, but dated on-or-before it, creates a restatement question the bank must be able to answer procedurally, not ad hoc.

One control before going live: mandatory reason-coded, dual-approved backdating. Any entry whose value_date differs from its actual posting day beyond a defined tolerance (say, T+1) must carry a structured reason code (late clearing, correction, dispute reversal, ...), require a second authorizer distinct from whoever posted it, and automatically land on a same-day exception report for reconciliation review. This doesn't stop backdating: value-dated correction is a legitimate, necessary banking operation; it converts it from a silent capability into a supervised, audited one, which is the actual gap in the current design (backdating works correctly; it just isn't watched).

3. Authorization lifecycle

A settlement is only one of several ways an authorization's life can end. Each of the following needs its own first-class, terminal state, not a special case bolted onto settlement or reversal, because each carries different money-movement and different downstream (dispute, support, regulatory) consequences:

- **Voluntary release/cancellation**, pre-settlement. *Real scenario:* a hotel pre-authorization voided at checkout because the guest paid by another method. *Mandated behavior:* a `RELEASED` status distinct from both `DECLINED` (never held funds) and `SETTLED` (funds actually moved); available balance is restored immediately, and support tooling needs to be able to answer "was this ever charged" precisely.
- **Expiry.** *Real scenario:* a rental-car pre-authorization the merchant never captures, and the network's hold window lapses. *Mandated behavior:* a scheduled sweep that automatically releases holds past their lifetime; without one, an un-swept hold is a permanent phantom debit against a customer's available funds, which is both a support and a regulatory-complaint vector. This cannot be a manual process.
- **Declined at authorization time** (this system's Auth-B). *Real scenario:* insufficient available balance at swipe time. *Mandated behavior:* recorded as its own terminal state, `DECLINED`, because it never held funds at all; conflating it with `RELEASED` makes "why did my available balance never move" unanswerable.
- **Partial settlement with release of the remainder.** *Real scenario:* a $200 fuel-pump hold that settles at $42 once the pump usage is known. *Mandated behavior:* the settled amount and the released remainder must both be tracked explicitly (not just "hold closed"); card-network dispute processes and reconciliation both need the split, not just the net effect.
- **Multiple partial captures against one authorization.** *Real scenario:* a hotel folio or a split-shipment order settling in several captures against the same original hold. This system deliberately does not support it: a hold is single-shot (`ACTIVE` $\rightarrow$ exactly one terminal state), which is a named cut below, not an oversight.
- **Issuer- or network-side unilateral release** (fraud hold, chargeback investigation). *Real scenario:* the card network reverses a hold outside the merchant/customer flow entirely. *Mandated behavior:* this has to be ingestible as its own authorization-lifecycle event type, because it originates outside the settlement/reversal vocabulary this system currently has.

4. What was cut, and the risk each cut defers

- **Single terminal settlement per authorization; no partial/multi-capture.** Defers the risk that a real merchant flow (hotel, car rental, tips, split shipment) legitimately needs several settlements against one hold; this system would reject that as "authorization not ACTIVE."

- **No time-based hold expiry.** Defers the risk that an un-settled, un-declined hold locks a customer's available funds forever with no automatic recovery; acceptable for a fixed six-day, fully-enumerated event stream, not acceptable for a live book.
- **No cap or review policy on over-settlement** (settlement amount exceeding the original hold). Defers real fraud/dispute exposure: this system books whatever a settlement event states once its authId is ACTIVE, with no re-authorization or tolerance check.
- **Overdraft fee defined only for AED.** Defers the fact that a live multi-currency ledger needs an explicit fee (or no-fee) policy for *every* currency it holds; this system surfaces that gap loudly (a warning, and a failing test) rather than silently doing nothing or guessing a number, but the gap itself is real and would block a BHD launch.
- **No persistence.** In scope by explicit assignment constraint; defers the obvious risk that a live ledger cannot lose all state on process restart. The append-only, ordered-entry design here is deliberately chosen so that adding a durable event log/snapshot store later is an additive change, not a rewrite.
- **No concurrency control.** Replay is single-threaded and synchronous; the authorization check ("is available balance still ≥ 0 after this hold") is a check-then-act with no lock around it. Defers a real time-of-check/time-of-use race between two authorizations landing on the same account at once: fine for a strictly ordered replay, not fine for concurrent production traffic.
- **No idempotency on event ids.** Replaying an event twice double-books it. Defers the reality that production ingestion (queues, retries) is typically at-least-once delivery, which needs a dedup check at the boundary before anything reaches this core.
- **No input validation at the ledger boundary.** Amounts, currencies, and precisions are trusted as already-correct. Defers the need for a real ingestion layer to reject malformed or hostile input before it reaches core ledger logic, which this exercise assumes is upstream of scope.